

Especificação Completa do Aplicativo — Meu Bicho de Volta

1. Visão geral do produto

O aplicativo **Meu Bicho de Volta** será uma plataforma mobile de comunidade para **achados e perdidos de pets**, permitindo que tutores cadastrem animais perdidos e notifiquem automaticamente usuários próximos ao local do desaparecimento.

A ideia central é criar uma rede colaborativa geolocalizada, onde cada usuário pode ajudar a encontrar pets perdidos na sua região. Quando um tutor cadastra um pet perdido, o sistema deve localizar todos os usuários dentro do raio definido e enviar uma notificação PUSH informando que existe um animalzinho perdido por perto. Essa notificação também deve ficar registrada dentro da caixa de mensagens/notificações internas do usuário.

O aplicativo deve ter:

- Aplicativo mobile para usuários finais.
- Painel administrativo web para gerenciamento da plataforma.
- Backend em arquitetura limpa, componentizada e preparada para microserviços.
- Serviços resilientes, escaláveis e de fácil manutenção.
- Comunicação via APIs bem definidas.
- Notificações PUSH e caixa interna de mensagens.
- Geolocalização por raio.
- Moderação de publicações.
- Recursos de segurança, privacidade e LGPD.

2. Objetivo do aplicativo

O objetivo do **Meu Bicho de Volta** é aumentar as chances de reencontro entre tutores e pets perdidos, usando tecnologia, geolocalização, notificações em tempo real e colaboração comunitária.

O aplicativo deve permitir que:

1. Um tutor cadastre rapidamente um pet perdido.
2. O tutor informe o local aproximado onde o pet foi perdido.
3. O tutor defina um raio de busca, inicialmente com padrão de **3 km**.
4. Usuários dentro desse raio sejam avisados por PUSH.
5. A notificação fique salva na central de mensagens internas.
6. Usuários possam visualizar detalhes do pet perdido.
7. Usuários possam informar que viram ou encontraram o animal.
8. O tutor receba essas informações rapidamente.
9. O administrador consiga moderar casos, usuários, denúncias e conteúdos.

3. Plataformas

3.1 Aplicativo mobile

O app do usuário deve ser **100% mobile**, com foco em Android e iOS.

Tecnologia sugerida:

- React Native com Expo e TypeScript
ou
- Flutter

Preferência de arquitetura do front-end:

- Componentização forte.
- Separação entre telas, componentes, hooks, serviços, estados e modelos.
- Design system próprio.
- Código limpo e fácil de manter.

3.2 Painel administrativo web

Deve existir um painel web exclusivo para o dono/admin da plataforma.

Tecnologia sugerida:

- React + TypeScript
 - Vite
 - Tailwind CSS
 - Shadcn/ui ou outro kit moderno de componentes
 - Consumo das mesmas APIs administrativas do backend
-

4. Personas principais

4.1 Tutor do pet perdido

Pessoa que perdeu um animal e precisa cadastrar rapidamente o caso no aplicativo.

Principais necessidades:

- Cadastro simples e rápido.
- Adicionar fotos do pet.
- Informar localização.
- Definir raio de busca.
- Receber avisos de pessoas que viram o pet.
- Encerrar o caso quando o animal for encontrado.

4.2 Usuário colaborador

Pessoa que usa o aplicativo para ajudar animais perdidos na região.

Principais necessidades:

- Receber alertas de pets perdidos próximos.
- Ver detalhes do animal.
- Avisar que viu o pet.
- Enviar localização aproximada.
- Enviar mensagem ou relato ao tutor.
- Salvar alertas importantes.

4.3 Pessoa que encontrou um pet

Usuário que encontrou um animal e deseja publicar um alerta de pet encontrado.

Principais necessidades:

- Cadastrar pet encontrado.
- Informar local onde encontrou.
- Adicionar fotos.
- Permitir que tutores entrem em contato.
- Receber possíveis correspondências com pets perdidos.

4.4 Administrador da plataforma

Dono ou operador da plataforma.

Principais necessidades:

- Moderar publicações.
- Visualizar casos ativos.
- Gerenciar usuários.
- Ver denúncias.
- Remover conteúdos impróprios.
- Acompanhar métricas.
- Enviar comunicados.
- Bloquear usuários mal-intencionados.

5. Funcionalidades principais do MVP

5.1 Autenticação

O app deve permitir cadastro e login de usuários.

Métodos de autenticação

- E-mail e senha.

- Login social com Google.
- Login social com Apple, especialmente para iOS.
- Recuperação de senha.
- Validação de e-mail.
- Aceite de termos de uso e política de privacidade.

Dados básicos do usuário

- Nome.
 - E-mail.
 - Telefone opcional.
 - Cidade.
 - Estado.
 - Foto de perfil opcional.
 - Permissão de localização.
 - Permissão de notificações PUSH.
-

5.2 Cadastro de pet perdido

O usuário deve conseguir cadastrar um pet perdido com rapidez.

Campos obrigatórios

- Nome do pet.
- Tipo do animal: cachorro, gato, ave, outro.
- Raça, quando souber.
- Porte: pequeno, médio, grande.
- Cor predominante.
- Fotos do pet.
- Data e hora aproximada do desaparecimento.
- Local onde foi visto pela última vez.
- Raio de busca.
- Informações importantes.
- Nome do tutor.
- Forma de contato.

Campos opcionais

- Idade.
- Sexo.
- Castrado ou não.
- Usa coleira?
- Possui identificação?
- Possui microchip?
- Temperamento.
- Responde por algum apelido?
- Recompensa.
- Observações médicas.
- Informações sobre medo, agressividade ou necessidade especial.

Raio de busca

O raio padrão inicial deve ser de **3 km**, mas o usuário poderá alterar dentro dos limites definidos pelo sistema.

Sugestão de limites:

- 1 km
- 3 km
- 5 km
- 10 km
- 20 km

No MVP, usar 3 km como padrão.

5.3 Notificação automática por localização

Quando um pet perdido for cadastrado, o backend deve:

1. Salvar o caso.
2. Validar a localização.
3. Buscar usuários ativos dentro do raio configurado.
4. Criar notificações internas para esses usuários.
5. Enviar PUSH via FCM/APNs.
6. Registrar evento para auditoria.
7. Permitir reenvio controlado em casos de atualização importante.

Texto padrão de PUSH

Título:

Animalzinho perdido perto de você

Mensagem:

Um pet foi perdido próximo da sua região. Toque para ver os detalhes e ajudar.

Texto interno da notificação

Existe um animalzinho perdido dentro de um raio próximo à sua localização. Veja os detalhes, compartilhe e avise o tutor caso tenha alguma informação.

5.4 Caixa de mensagens / notificações internas

O app deve possuir uma área chamada **Notificações** ou **Caixa de Mensagens**.

Essa área deve listar:

- Alertas de pets perdidos próximos.
- Atualizações de casos acompanhados.
- Respostas de tutores.
- Relatos enviados ao tutor.
- Avisos administrativos.
- Status de caso encontrado.
- Notificações de moderação.

Cada notificação deve conter:

- Título.
- Mensagem curta.
- Foto do pet, quando aplicável.
- Data e hora.
- Status lida/não lida.
- Link para o caso.
- Tipo da notificação.

Tipos sugeridos:

- PET_LOST_NEARBY
- CASE_UPDATED
- PET_FOUND
- SIGHTING_RECEIVED
- MESSAGE_RECEIVED
- ADMIN_NOTICE
- MODERATION_NOTICE

5.5 Feed de pets perdidos próximos

Tela principal do app mostrando pets perdidos próximos ao usuário.

Filtros

- Distância.
- Tipo do animal.
- Data de desaparecimento.
- Status do caso.
- Cachorro.
- Gato.
- Outros.
- Casos com recompensa.
- Casos urgentes.

Ordenação

- Mais próximos.
- Mais recentes.

- Mais compartilhados.
 - Mais urgentes.
-

5.6 Mapa de pets perdidos

O app deve ter uma visualização em mapa.

No mapa devem aparecer:

- Pets perdidos.
- Pets encontrados.
- Últimos locais avistados.
- Área aproximada do raio de busca.

Importante: por privacidade, a localização exata do tutor não deve ser exibida. Exibir apenas o local do desaparecimento ou ponto aproximado.

5.7 Detalhe do caso

Ao abrir um pet perdido, o usuário deve ver:

- Fotos do pet.
 - Nome.
 - Tipo.
 - Raça.
 - Cor.
 - Porte.
 - Local aproximado onde desapareceu.
 - Distância até o usuário.
 - Data/hora do desaparecimento.
 - Descrição.
 - Temperamento.
 - Informações médicas.
 - Dados de contato permitidos pelo tutor.
 - Botão "Vi esse pet".
 - Botão "Encontrei esse pet".
 - Botão "Compartilhar".
 - Botão "Salvar caso".
 - Botão "Denunciar".
-

5.8 Relato de avistamento

Usuários devem conseguir informar que viram o pet.

Campos:

- Data e hora em que viu.
- Local aproximado.
- Comentário.
- Foto opcional.
- Permitir compartilhar localização atual.
- Nível de certeza: baixa, média, alta.

Após o envio:

- O tutor recebe PUSH.
 - O tutor recebe notificação interna.
 - O relato aparece na linha do tempo do caso.
 - O admin pode auditar relatos suspeitos.
-

5.9 Fluxo “Encontrei esse pet”

Quando um usuário encontrar um pet, ele deve poder enviar um alerta mais forte para o tutor.

Campos:

- Foto do pet encontrado.
- Local aproximado.
- Mensagem.
- Telefone de contato opcional.
- Disponibilidade para contato.
- Botão para iniciar conversa segura.

O tutor deve receber uma notificação prioritária.

5.10 Chat seguro entre tutor e colaborador

O app pode ter um chat interno simples.

No MVP, o chat pode ser limitado por caso.

Regras:

- O chat só existe vinculado a um caso.
 - O colaborador não precisa ver todos os dados pessoais do tutor.
 - O tutor pode encerrar a conversa.
 - O usuário pode bloquear ou denunciar.
 - As mensagens devem ser auditáveis pelo admin em caso de denúncia.
-

5.11 Compartilhamento do caso

O app deve gerar um card de compartilhamento para WhatsApp, Instagram e outras redes.

O card deve conter:

- Foto do pet.
- Nome.
- Texto “Pet perdido”.
- Bairro/cidade aproximada.
- Data do desaparecimento.
- Link público do caso.
- QR Code opcional.

Texto sugerido:

Ajude a encontrar o(a) [NOME DO PET]. Foi visto pela última vez próximo de [LOCAL]. Acesse o link para ver detalhes e avisar o tutor.

5.12 Página pública do caso

Cada caso deve ter uma página pública acessível por link.

Essa página deve permitir que pessoas sem app vejam informações básicas e enviem uma pista.

Conteúdo:

- Foto.
 - Nome do pet.
 - Local aproximado.
 - Descrição.
 - Botão “Tenho uma informação”.
 - Botão para abrir no aplicativo.
 - Botão de compartilhamento.
-

5.13 Cadastro de pet encontrado

Além de pet perdido, o app deve permitir cadastrar **pet encontrado**.

Campos:

- Tipo de animal.
- Fotos.
- Local onde foi encontrado.
- Data/hora.
- Características.
- Se está com coleira.

- Se está machucado.
- Se foi levado para algum local seguro.
- Contato do usuário que encontrou.

O sistema pode sugerir possíveis correspondências com pets perdidos próximos.

5.14 Correspondência automática entre perdido e encontrado

O sistema deve cruzar casos de pets perdidos com pets encontrados.

CrITÉrios:

- Distância geográfica.
- Tipo do animal.
- Cor.
- Porte.
- Raça.
- Data.
- Palavras-chave na descrição.

Quando houver possível correspondência:

- Notificar o tutor.
- Notificar quem cadastrou o pet encontrado.
- Exibir sugestão no painel admin.
- Permitir confirmação manual.

No MVP, essa correspondência pode ser simples por filtros e localização. Em versões futuras, pode usar IA para comparação visual de fotos.

5.15 Status do caso

Cada caso deve possuir status.

Status sugeridos:

- DRAFT
- PENDING_REVIEW
- ACTIVE
- PAUSED
- FOUND
- CANCELLED
- EXPIRED
- REJECTED
- REMOVED_BY_ADMIN

No MVP, o fluxo pode ser:

1. Usuário cadastra.

2. Caso fica ACTIVE automaticamente ou PENDING_REVIEW, conforme configuração.
 3. Usuários próximos são notificados.
 4. Tutor recebe relatos.
 5. Tutor marca como FOUND quando encontrar.
 6. Sistema avisa pessoas que acompanharam o caso.
-

5.16 Linha do tempo do caso

Cada caso deve ter uma linha do tempo.

Eventos possíveis:

- Caso criado.
 - Caso aprovado.
 - Alerta enviado.
 - Usuário relatou avistamento.
 - Usuário informou que encontrou.
 - Tutor atualizou descrição.
 - Tutor alterou raio de busca.
 - Caso compartilhado.
 - Caso marcado como encontrado.
 - Caso encerrado.
-

5.17 Denúncias e moderação

Usuários podem denunciar:

- Caso falso.
- Foto imprópria.
- Golpe.
- Informação incorreta.
- Usuário abusivo.
- Spam.
- Conteúdo sensível.

O admin deve conseguir:

- Ver denúncias.
 - Suspendar caso.
 - Remover caso.
 - Bloquear usuário.
 - Enviar aviso.
 - Marcar denúncia como resolvida.
-

6. Funcionalidades do painel administrativo web

O painel admin deve conter:

6.1 Login administrativo

- Login com e-mail e senha.
- Permissões por perfil.
- Sessão segura.
- MFA opcional.

Perfis:

- SUPER_ADMIN
 - ADMIN
 - MODERATOR
 - SUPPORT
-

6.2 Dashboard

Exibir métricas:

- Total de usuários.
 - Usuários ativos.
 - Pets perdidos ativos.
 - Pets encontrados.
 - Casos resolvidos.
 - Casos denunciados.
 - Alertas enviados.
 - Taxa de reencontro.
 - Cidades com mais casos.
 - Tempo médio até encontrar.
 - Quantidade de notificações enviadas.
-

6.3 Gerenciamento de casos

O admin deve conseguir:

- Listar todos os casos.
- Filtrar por status.
- Filtrar por cidade.
- Filtrar por data.
- Abrir detalhe do caso.
- Aprovar caso.
- Rejeitar caso.
- Remover caso.
- Marcar como destaque.

- Alterar status.
 - Ver linha do tempo.
 - Ver relatos recebidos.
 - Ver denúncias.
-

6.4 Gerenciamento de usuários

O admin deve conseguir:

- Listar usuários.
 - Ver dados básicos.
 - Ver casos criados.
 - Ver denúncias recebidas.
 - Bloquear usuário.
 - Desbloquear usuário.
 - Marcar usuário como suspeito.
 - Remover acesso.
-

6.5 Moderação de denúncias

Tela específica para denúncias.

Campos:

- Tipo da denúncia.
- Usuário denunciante.
- Usuário denunciado.
- Caso relacionado.
- Mensagem.
- Status.
- Data.
- Ações tomadas.

Status:

- OPEN
 - IN_REVIEW
 - RESOLVED
 - DISMISSED
-

6.6 Comunicados administrativos

O admin pode enviar comunicados para:

- Todos os usuários.
- Usuários de uma cidade.

- Usuários dentro de um raio.
- Usuários que acompanham um caso.

Tipos:

- PUSH.
 - Notificação interna.
 - Ambos.
-

6.7 Configurações do sistema

O painel deve permitir configurar:

- Raio padrão.
 - Raio máximo permitido.
 - Tempo de expiração de casos.
 - Moderação automática ligada/desligada.
 - Limite de casos por usuário.
 - Limite de notificações por dia.
 - Textos padrão.
 - Termos de uso.
 - Política de privacidade.
-

7. Melhorias importantes para o app

7.1 QR Code público do pet

Cada pet cadastrado pode ter um QR Code único.

Esse QR Code pode apontar para uma página pública do pet ou do caso.

Uso futuro:

- Plaquinhas físicas.
 - Coleiras.
 - Produtos personalizados.
 - Integração com NFC.
-

7.2 NFC ou tag física

Em uma versão futura, o app pode permitir vincular uma tag NFC ao pet.

Ao escanear a tag:

- Abre a página do pet.
- Mostra contato seguro.

- Notifica o tutor que alguém escaneou.
 - Mostra localização aproximada do escaneamento, se permitido.
-

7.3 Modo comunidade

Usuários podem se tornar “anjos da região” ou voluntários.

Recursos:

- Ranking de ajuda.
 - Casos ajudados.
 - Selos de colaboração.
 - Histórico de avistamentos úteis.
-

7.4 Campanhas pagas opcionais

No futuro, o tutor pode impulsionar o alerta.

Exemplos:

- Destacar caso por 24h.
- Enviar alerta para raio maior.
- Gerar cartaz premium.
- Enviar alerta recorrente.
- Criar anúncio patrocinado.

Essa funcionalidade não deve ser obrigatória no MVP.

7.5 Cartaz automático

O app pode gerar um cartaz em PDF/imagem para impressão.

Conteúdo:

- Foto do pet.
 - Nome.
 - “PROCURA-SE”
 - Bairro/cidade.
 - Telefone ou contato seguro.
 - QR Code do caso.
-

7.6 IA para comparação de imagens

Versão futura:

- Comparar foto de pet perdido com foto de pet encontrado.
 - Sugerir possíveis matches.
 - Apontar grau de semelhança.
 - Não tomar decisão automática, apenas sugerir.
-

8. Arquitetura geral

A arquitetura deve ser baseada em:

- Front-end mobile.
 - Front-end web admin.
 - API Gateway.
 - Microserviços independentes.
 - Banco relacional com suporte geoespacial.
 - Mensageria assíncrona.
 - Serviço de notificações.
 - Armazenamento de imagens.
 - Cache.
 - Observabilidade.
-

9. Stack técnica sugerida

Mobile

- React Native
- Expo
- TypeScript
- Zustand ou Redux Toolkit para estado global
- React Query para dados remotos
- Expo Notifications
- Expo Location
- React Navigation
- Zod para validações
- Design system próprio

Web admin

- React
- TypeScript
- Vite
- Tailwind CSS
- React Query
- React Hook Form
- Zod

- Shadcn/ui
- TanStack Table
- Recharts

Backend

- Node.js
- NestJS
- TypeScript
- Arquitetura limpa
- REST APIs inicialmente
- WebSocket para eventos em tempo real, quando necessário
- Microserviços com comunicação via fila/eventos

Banco de dados

- PostgreSQL
- PostGIS para geolocalização
- Redis para cache e filas leves
- Prisma ORM ou TypeORM

Mensageria

- RabbitMQ
ou
- AWS SQS/SNS
ou
- Kafka em uma versão mais robusta

Arquivos

- AWS S3 ou storage compatível
- CloudFront/CDN para imagens

PUSH

- Firebase Cloud Messaging para Android
- APNs para iOS
- Expo Push Notifications, se usar Expo no MVP

Infra

- Docker
 - Docker Compose para ambiente local
 - Kubernetes ou ECS para produção futura
 - CI/CD com GitHub Actions
 - Logs centralizados
 - Monitoramento com Prometheus/Grafana ou serviço equivalente
-

10. Microserviços sugeridos

10.1 API Gateway

Responsabilidades:

- Receber chamadas dos aplicativos.
- Validar autenticação.
- Fazer roteamento para microserviços.
- Rate limiting.
- Logs de entrada.
- Versionamento de API.

Rotas principais:

- `/api/v1/auth`
 - `/api/v1/users`
 - `/api/v1/pets`
 - `/api/v1/lost-cases`
 - `/api/v1/found-cases`
 - `/api/v1/notifications`
 - `/api/v1/messages`
 - `/api/v1/admin`
-

10.2 Auth Service

Responsabilidades:

- Cadastro.
- Login.
- Refresh token.
- Recuperação de senha.
- Autorização por perfil.
- Controle de sessão.
- Integração com login social.

Entidades:

- User
 - AuthSession
 - Role
 - Permission
-

10.3 User Service

Responsabilidades:

- Perfil do usuário.
- Preferências.
- Permissão de localização.
- Permissão de PUSH.
- Cidade/estado.
- Device tokens.
- Bloqueios.

Entidades:

- UserProfile
 - UserDevice
 - UserLocation
 - UserPreference
-

10.4 Pet Service

Responsabilidades:

- Cadastro de pets do usuário.
- Dados fixos do animal.
- Fotos.
- QR Code.
- Tags futuras.
- Histórico do pet.

Entidades:

- Pet
 - PetPhoto
 - PetTag
-

10.5 Lost Case Service

Responsabilidades:

- Cadastro de pet perdido.
- Controle de status.
- Linha do tempo.
- Relatos de avistamento.
- Atualizações.
- Encerramento do caso.

Entidades:

- LostPetCase
 - CaseTimelineEvent
 - SightingReport
 - CaseFollower
-

10.6 Found Case Service

Responsabilidades:

- Cadastro de pet encontrado.
- Busca de possíveis correspondências.
- Sugestões para tutores.
- Controle de status.

Entidades:

- FoundPetCase
 - PossibleMatch
-

10.7 Geo Service

Responsabilidades:

- Consultas geográficas.
- Buscar usuários dentro de raio.
- Buscar casos próximos.
- Calcular distância.
- Anonimizar localização.
- Gerenciar índice geoespacial.

Tecnologia:

- PostgreSQL + PostGIS.

Funções principais:

- `findUsersWithinRadius(latitude, longitude, radiusKm)`
 - `findCasesNearUser(userId, radiusKm)`
 - `calculateDistance(pointA, pointB)`
 - `maskExactLocation(location)`
-

10.8 Notification Service

Responsabilidades:

- Criar notificações internas.
- Enviar PUSH.
- Controlar leitura.
- Controlar tentativas.
- Evitar spam.
- Registrar falhas.

Entidades:

- Notification
- NotificationDelivery
- PushToken

Eventos consumidos:

- `lost_case.created`
- `lost_case.updated`
- `sighting.created`
- `case.marked_found`
- `admin.notice.created`

10.9 Messaging Service

Responsabilidades:

- Chat interno.
- Conversas vinculadas a casos.
- Mensagens entre tutor e colaborador.
- Bloqueios.
- Denúncias de conversa.

Entidades:

- Conversation
- Message
- MessageAttachment

10.10 Media Service

Responsabilidades:

- Upload de imagens.
- Compressão.
- Validação.

- Armazenamento.
- Remoção segura.
- Geração de URLs assinadas.
- Controle de conteúdo.

Entidades:

- MediaFile
-

10.11 Moderation Service

Responsabilidades:

- Denúncias.
- Revisão de casos.
- Bloqueio de conteúdo.
- Histórico de decisões administrativas.

Entidades:

- Report
 - ModerationAction
 - UserPenalty
-

10.12 Admin Service

Responsabilidades:

- Dashboard.
- Métricas.
- Gerenciamento administrativo.
- Configurações globais.
- Comunicados.

Entidades:

- SystemSetting
 - AdminAuditLog
 - AdminNotice
-

11. Comunicação entre serviços

11.1 Comunicação síncrona

Usar REST para operações simples:

- Buscar perfil.

- Criar caso.
- Listar pets.
- Atualizar status.
- Consultar notificações.

11.2 Comunicação assíncrona

Usar eventos para processos que não precisam travar a experiência do usuário.

Exemplo: ao criar um caso perdido, o app não deve esperar todo o envio de notificações terminar.

Fluxo correto:

1. Usuário cria caso.
2. Lost Case Service salva o caso.
3. Lost Case Service publica evento `lost_case.created`.
4. Notification Service consome o evento.
5. Geo Service identifica usuários no raio.
6. Notification Service cria notificações internas.
7. Notification Service envia PUSH.
8. Logs e tentativas são registrados.

12. Eventos do sistema

Eventos principais:

```
user.created
user.location.updated
pet.created
pet.updated
lost_case.created
lost_case.updated
lost_case.marked_found
lost_case.expired
found_case.created
possible_match.created
sighting.created
notification.created
notification.sent
notification.failed
message.created
report.created
moderation.action_applied
admin.notice.created
```

13. Modelo de dados conceitual

13.1 User

```
id
name
email
phone
password_hash
role
status
profile_photo_url
city
state
created_at
updated_at
last_login_at
```

Status:

```
ACTIVE
BLOCKED
DELETED
PENDING_EMAIL_VALIDATION
```

13.2 UserDevice

```
id
user_id
device_id
platform
push_token
push_enabled
last_seen_at
created_at
updated_at
```

Platform:

```
ANDROID
IOS
WEB
```


13.3 UserLocation

```
id
user_id
latitude
longitude
city
state
source
updated_at
```

Source:

```
GPS
MANUAL
APPROXIMATE
```

13.4 Pet

```
id
owner_id
name
type
breed
size
color
sex
age
is_neutered
has_microchip
medical_notes
temperament
photo_cover_url
created_at
updated_at
```

13.5 PetPhoto

```
id
pet_id
url
```

```
order
created_at
```

13.6 LostPetCase

```
id
pet_id
owner_id
status
title
description
last_seen_at
last_seen_latitude
last_seen_longitude
last_seen_address_text
radius_km
reward_enabled
reward_description
contact_preference
urgency_level
created_at
updated_at
resolved_at
expires_at
```

13.7 FoundPetCase

```
id
created_by_user_id
status
animal_type
description
found_at
found_latitude
found_longitude
found_address_text
photo_cover_url
contact_preference
created_at
updated_at
resolved_at
```

13.8 SightingReport

```
id
lost_case_id
reported_by_user_id
seen_at
latitude
longitude
address_text
certainty_level
comment
photo_url
created_at
```

Certainty:

```
LOW
MEDIUM
HIGH
```

13.9 Notification

```
id
user_id
type
title
body
related_entity_type
related_entity_id
is_read
created_at
read_at
```

13.10 NotificationDelivery

```
id
notification_id
channel
status
attempts
provider_message_id
error_message
```

```
sent_at
created_at
```

Channel:

```
PUSH
IN_APP
EMAIL
SMS
```

Status:

```
PENDING
SENT
FAILED
RETRYING
CANCELLED
```

13.11 Conversation

```
id
case_id
case_type
owner_user_id
participant_user_id
status
created_at
updated_at
```

13.12 Message

```
id
conversation_id
sender_user_id
body
attachment_url
is_read
created_at
```

13.13 Report

```
id
created_by_user_id
target_type
target_id
reason
description
status
created_at
resolved_at
```

13.14 SystemSetting

```
id
key
value
description
updated_by_admin_id
updated_at
```

14. Regras de negócio

14.1 Criação de caso perdido

- O usuário precisa estar autenticado.
 - O usuário precisa aceitar os termos.
 - O usuário precisa informar ao menos uma foto.
 - O raio padrão é 3 km.
 - O raio máximo deve respeitar configuração do sistema.
 - O caso deve receber status ACTIVE ou PENDING_REVIEW.
 - O sistema deve criar evento na timeline.
 - O sistema deve disparar evento para notificações.
 - O usuário criador não deve receber notificação do próprio caso como alerta próximo.
-

14.2 Notificação por raio

- O sistema deve buscar usuários com localização conhecida.
- Usuários bloqueados não recebem alertas.
- Usuários com PUSH desativado recebem apenas notificação interna.
- Usuários sem localização não entram no disparo por raio.
- O sistema deve evitar notificar o mesmo usuário repetidamente sobre o mesmo caso.
- O sistema deve registrar falhas de envio.

- O envio de PUSH deve ser assíncrono.
-

14.3 Privacidade de localização

- A localização exata do tutor não deve ser exibida.
 - A localização do desaparecimento pode ser exibida de forma aproximada.
 - O usuário deve autorizar uso de localização.
 - O usuário pode usar localização manual.
 - O app deve explicar por que precisa da localização.
 - Dados sensíveis devem ser protegidos conforme LGPD.
-

14.4 Relatos de avistamento

- Apenas usuários autenticados podem enviar relatos.
 - O tutor deve ser notificado.
 - Relatos suspeitos podem ser denunciados.
 - O admin pode remover relatos.
 - O relato deve ficar vinculado ao caso.
-

14.5 Encerramento de caso

- Apenas o tutor ou admin pode encerrar um caso.
 - Ao marcar como encontrado, o sistema deve:
 - Atualizar status para FOUND.
 - Registrar evento na timeline.
 - Notificar usuários que acompanharam o caso.
 - Remover o caso dos alertas ativos.
 - Manter histórico.
-

14.6 Expiração automática

- Casos podem expirar após determinado período.
 - Sugestão inicial: 30 dias.
 - Antes de expirar, enviar aviso ao tutor.
 - O tutor pode renovar o caso.
 - Casos expirados não aparecem no feed ativo.
-

15. APIs sugeridas

15.1 Auth

```
POST /api/v1/auth/register
POST /api/v1/auth/login
POST /api/v1/auth/logout
POST /api/v1/auth/refresh-token
POST /api/v1/auth/forgot-password
POST /api/v1/auth/reset-password
GET /api/v1/auth/me
```

15.2 Usuários

```
GET /api/v1/users/me
PATCH /api/v1/users/me
PATCH /api/v1/users/me/location
POST /api/v1/users/me/devices
DELETE /api/v1/users/me/devices/:deviceId
PATCH /api/v1/users/me/preferences
```

15.3 Pets

```
POST /api/v1/pets
GET /api/v1/pets/me
GET /api/v1/pets/:id
PATCH /api/v1/pets/:id
DELETE /api/v1/pets/:id
POST /api/v1/pets/:id/photos
DELETE /api/v1/pets/:id/photos/:photoId
```

15.4 Casos de pet perdido

```
POST /api/v1/lost-cases
GET /api/v1/lost-cases/nearby
GET /api/v1/lost-cases/me
GET /api/v1/lost-cases/:id
PATCH /api/v1/lost-cases/:id
POST /api/v1/lost-cases/:id/mark-found
POST /api/v1/lost-cases/:id/pause
POST /api/v1/lost-cases/:id/renew
```

```
POST /api/v1/lost-cases/:id/follow
DELETE /api/v1/lost-cases/:id/follow
```

15.5 Relatos de avistamento

```
POST /api/v1/lost-cases/:id/sightings
GET /api/v1/lost-cases/:id/sightings
GET /api/v1/sightings/me
```

15.6 Pets encontrados

```
POST /api/v1/found-cases
GET /api/v1/found-cases/nearby
GET /api/v1/found-cases/me
GET /api/v1/found-cases/:id
PATCH /api/v1/found-cases/:id
POST /api/v1/found-cases/:id/resolve
```

15.7 Notificações

```
GET /api/v1/notifications
GET /api/v1/notifications/unread-count
PATCH /api/v1/notifications/:id/read
PATCH /api/v1/notifications/read-all
```

15.8 Mensagens

```
POST /api/v1/conversations
GET /api/v1/conversations
GET /api/v1/conversations/:id
POST /api/v1/conversations/:id/messages
PATCH /api/v1/conversations/:id/read
```


15.9 Denúncias

```
POST /api/v1/reports
GET /api/v1/reports/me
```

15.10 Admin

```
GET /api/v1/admin/dashboard
GET /api/v1/admin/users
GET /api/v1/admin/users/:id
PATCH /api/v1/admin/users/:id/block
PATCH /api/v1/admin/users/:id/unblock

GET /api/v1/admin/lost-cases
GET /api/v1/admin/lost-cases/:id
PATCH /api/v1/admin/lost-cases/:id/status

GET /api/v1/admin/found-cases
GET /api/v1/admin/reports
PATCH /api/v1/admin/reports/:id/status

GET /api/v1/admin/settings
PATCH /api/v1/admin/settings

POST /api/v1/admin/notices
```

16. Telas do aplicativo mobile

16.1 Splash

- Logo Meu Bicho de Volta.
- Carregamento inicial.
- Verificação de sessão.

16.2 Onboarding

Explicar:

- O app ajuda pets perdidos a voltarem para casa.
- Você recebe alertas de animais perdidos perto de você.
- Sua localização é usada apenas para alertas relevantes.
- Você pode ajudar enviando pistas.
- Solicitar permissões de localização e PUSH.

16.3 Login

- Entrar com e-mail e senha.
 - Entrar com Google.
 - Entrar com Apple.
 - Esqueci minha senha.
 - Criar conta.
-

16.4 Cadastro

- Nome.
 - E-mail.
 - Senha.
 - Cidade/estado.
 - Aceite de termos.
 - Permissões.
-

16.5 Home

Elementos:

- Saudação.
 - Botão principal: “Perdi meu pet”.
 - Botão secundário: “Encontrei um pet”.
 - Lista de pets perdidos próximos.
 - Cards com foto, nome, distância e data.
 - Atalho para mapa.
 - Atalho para notificações.
-

16.6 Feed próximo

- Listagem de casos.
 - Filtros.
 - Ordenação.
 - Busca por cidade/bairro.
-

16.7 Mapa

- Pins no mapa.
 - Filtro por tipo.
 - Exibição de raio aproximado.
 - Cards resumidos ao tocar no pin.
-

16.8 Cadastro de pet perdido

Etapas:

1. Dados do pet.
 2. Fotos.
 3. Último local visto.
 4. Raio de busca.
 5. Informações adicionais.
 6. Revisão.
 7. Publicar alerta.
-

16.9 Cadastro de pet encontrado

Etapas:

1. Fotos.
 2. Características.
 3. Local encontrado.
 4. Informações adicionais.
 5. Publicar.
-

16.10 Detalhe do caso

- Galeria.
 - Dados do pet.
 - Local aproximado.
 - Botões de ação.
 - Linha do tempo.
 - Relatos.
 - Compartilhar.
-

16.11 Enviar relato

- Informar local.
 - Comentário.
 - Foto opcional.
 - Grau de certeza.
 - Enviar.
-

16.12 Conversas

- Lista de conversas.
- Chat vinculado ao caso.

- Denunciar conversa.
 - Bloquear usuário.
-

16.13 Notificações

- Lista de alertas.
 - Lidas/não lidas.
 - Abrir caso.
 - Marcar todas como lidas.
-

16.14 Meus casos

- Casos perdidos cadastrados.
 - Casos encontrados cadastrados.
 - Status de cada caso.
 - Ações rápidas:
 - Editar.
 - Renovar.
 - Marcar como encontrado.
 - Pausar.
 - Compartilhar.
-

16.15 Perfil

- Dados pessoais.
 - Preferências de notificação.
 - Localização.
 - Meus pets.
 - Termos.
 - Política de privacidade.
 - Sair.
-

17. Componentização do front-end mobile

Estrutura sugerida:

```
src/  
  app/  
    navigation/  
    providers/  
    routes/  
  modules/  
    auth/  
    screens/
```

```
components/  
services/  
hooks/  
schemas/  
pets/  
  screens/  
  components/  
  services/  
  hooks/  
  schemas/  
lostCases/  
  screens/  
  components/  
  services/  
  hooks/  
  schemas/  
foundCases/  
  screens/  
  components/  
  services/  
  hooks/  
  schemas/  
notifications/  
messages/  
profile/  
map/  
shared/  
  components/  
    Button/  
    Input/  
    Card/  
    Avatar/  
    EmptyState/  
    LoadingState/  
    ErrorState/  
    BottomSheet/  
    Modal/  
  designSystem/  
    colors.ts  
    typography.ts  
    spacing.ts  
    radius.ts  
  services/  
    apiClient.ts  
    storage.ts  
    location.ts  
    push.ts  
  utils/  
  types/
```

18. Design system

Estilo visual:

- Amigável.
- Seguro.
- Emocional.
- Limpo.
- Mobile-first.
- Fácil para qualquer pessoa usar em momento de urgência.

Sugestão de cores:

- Primária: laranja ou vermelho alaranjado.
- Secundária: azul suave ou verde esperança.
- Fundo: branco/cinza claro.
- Texto: cinza escuro.
- Alertas: vermelho suave.
- Sucesso: verde.

Sugestão:

```
primary: #FF6F00  
primaryDark: #D85C00  
secondary: #2E7D32  
danger: #D32F2F  
background: #FAFAFA  
surface: #FFFFFF  
textPrimary: #212121  
textSecondary: #757575
```

Tipografia:

- Poppins ou Montserrat.

Componentes base:

- Button
- Input
- TextArea
- Select
- Radio
- Checkbox
- PetCard
- CaseCard
- NotificationCard
- MapMarker
- Avatar
- Badge
- BottomSheet

- ImageUploader
 - Stepper
 - EmptyState
 - LoadingState
 - ErrorState
-

19. Resiliência e performance

O sistema deve ser resiliente.

Requisitos:

- Envio de PUSH assíncrono.
 - Fila para processamento de notificações.
 - Retry automático em falhas.
 - Dead-letter queue para mensagens com erro.
 - Rate limit por usuário.
 - Cache para consultas frequentes.
 - Paginação em listas.
 - Compressão de imagens.
 - Logs estruturados.
 - Monitoramento de erros.
 - Health checks.
 - Idempotência em eventos críticos.
 - Tratamento offline parcial no app.
-

20. Funcionalidade offline parcial

O app deve lidar bem com internet ruim.

Comportamentos:

- Mostrar últimos casos carregados.
 - Permitir rascunho de cadastro de pet perdido.
 - Tentar upload novamente quando a internet voltar.
 - Exibir mensagens claras em caso de erro.
 - Não perder dados digitados em formulários longos.
-

21. Segurança

Requisitos:

- JWT com refresh token.
- Senhas com hash seguro.
- HTTPS obrigatório.
- Rate limit.

- Validação de entrada com Zod/class-validator.
 - Sanitização de textos.
 - Controle de acesso por perfil.
 - Proteção contra abuso de notificações.
 - Auditoria administrativa.
 - Upload seguro de imagens.
 - Limite de tamanho de arquivo.
 - Verificação de tipo de arquivo.
 - URLs assinadas quando necessário.
-

22. LGPD e privacidade

O sistema deve:

- Solicitar consentimento de localização.
 - Explicar uso da localização.
 - Permitir exclusão de conta.
 - Permitir exclusão de dados pessoais.
 - Não expor endereço exato do tutor.
 - Permitir ocultar telefone.
 - Usar chat interno como contato seguro.
 - Manter política de privacidade.
 - Registrar aceite de termos.
 - Minimizar dados coletados.
-

23. Observabilidade

O backend deve ter:

- Logs estruturados.
- Trace ID por requisição.
- Métricas por serviço.
- Monitoramento de filas.
- Monitoramento de envio de PUSH.
- Alertas de erro.
- Dashboard técnico.

Métricas importantes:

- Quantidade de casos criados.
 - Tempo médio de envio das notificações.
 - Taxa de falha de PUSH.
 - Usuários notificados por caso.
 - Tempo de resposta das APIs.
 - Erros por endpoint.
 - Volume de uploads.
 - Casos marcados como encontrados.
-

24. Testes

Tipos de teste:

- Testes unitários.
- Testes de integração.
- Testes end-to-end.
- Testes de componentes mobile.
- Testes de API.
- Testes de carga para notificações.
- Testes de geolocalização.
- Testes de permissão de localização/PUSH.
- Testes de segurança básicos.

Casos críticos:

- Criar caso perdido.
- Buscar usuários por raio.
- Enviar PUSH.
- Criar notificação interna.
- Marcar caso como encontrado.
- Enviar relato.
- Bloquear usuário.
- Denunciar conteúdo.
- Expirar caso.
- Renovar caso.

25. Fluxo principal de criação de caso perdido

```
Usuário abre app
→ toca em "Perdi meu pet"
→ seleciona ou cadastra pet
→ adiciona fotos
→ informa último local visto
→ define raio de busca, padrão 3 km
→ adiciona descrição
→ revisa os dados
→ publica o caso
→ backend salva caso
→ backend publica evento lost_case.created
→ Notification Service processa evento
→ Geo Service busca usuários dentro do raio
→ sistema cria notificações internas
→ sistema envia PUSH
→ usuários próximos recebem alerta
→ usuários acessam detalhe
→ usuários enviam relatos
→ tutor recebe relatos
```

- tutor encontra pet
- tutor marca caso como encontrado
- sistema encerra caso e avisa seguidores

26. Critérios de aceite do MVP

O MVP será considerado pronto quando:

1. Usuário conseguir criar conta e fazer login.
2. Usuário conseguir cadastrar um pet perdido.
3. Usuário conseguir informar localização e raio de busca.
4. Sistema conseguir encontrar usuários próximos.
5. Usuários próximos receberem notificação interna.
6. Usuários próximos receberem PUSH.
7. Usuário conseguir visualizar pets perdidos próximos.
8. Usuário conseguir enviar relato de avistamento.
9. Tutor conseguir receber relato.
10. Tutor conseguir marcar pet como encontrado.
11. Usuário conseguir cadastrar pet encontrado.
12. Admin conseguir acessar painel web.
13. Admin conseguir visualizar casos.
14. Admin conseguir moderar denúncias.
15. Sistema respeitar permissões de localização.
16. Sistema não expor localização exata indevidamente.
17. App funcionar de forma estável em Android e iOS.

27. Roadmap sugerido

Fase 1 — MVP essencial

- Login/cadastro.
- Perfil do usuário.
- Permissão de localização.
- Cadastro de pet perdido.
- Cadastro de pet encontrado.
- Feed de pets próximos.
- Notificação interna.
- PUSH.
- Relato de avistamento.
- Marcar como encontrado.
- Painel admin básico.

Fase 2 — Comunidade e confiança

- Chat interno.
- Denúncias.
- Moderação completa.

- Linha do tempo do caso.
- Compartilhamento com card.
- Página pública do caso.
- Cartaz automático.

Fase 3 — Recursos avançados

- QR Code do pet.
- Tag NFC.
- Match automático entre perdido/encontrado.
- Ranking de voluntários.
- Campanhas pagas.
- IA para comparação de imagens.
- Estatísticas públicas por região.

28. Prompt para a IA desenvolvedora

Desenvolva o aplicativo **Meu Bicho de Volta**, uma plataforma mobile de achados e perdidos de pets baseada em geolocalização, notificações PUSH e colaboração comunitária.

O sistema deve ter:

1. App mobile para usuários finais.
2. Painel administrativo web.
3. Backend com arquitetura limpa e preparada para microserviços.
4. Banco PostgreSQL com PostGIS.
5. Notificações PUSH e notificações internas.
6. Upload de imagens.
7. Geolocalização por raio.
8. Fluxos para pet perdido e pet encontrado.
9. Moderação administrativa.
10. Código componentizado, resiliente e fácil de entender.

Use preferencialmente:

- React Native + Expo + TypeScript no mobile.
- React + Vite + TypeScript + Tailwind no painel admin.
- NestJS + TypeScript no backend.
- PostgreSQL + PostGIS.
- Redis para cache.
- RabbitMQ ou SQS para mensageria.
- S3 ou storage compatível para imagens.
- Firebase Cloud Messaging ou Expo Notifications para PUSH.

O app mobile deve conter:

- Splash.
- Onboarding.
- Login.
- Cadastro.

- Home.
- Feed de pets próximos.
- Mapa.
- Cadastro de pet perdido.
- Cadastro de pet encontrado.
- Detalhe do caso.
- Envio de relato de avistamento.
- Conversas.
- Notificações internas.
- Meus casos.
- Perfil.

O painel admin deve conter:

- Login administrativo.
- Dashboard.
- Gerenciamento de usuários.
- Gerenciamento de casos perdidos.
- Gerenciamento de pets encontrados.
- Moderação de denúncias.
- Configurações do sistema.
- Comunicados administrativos.

Regras fundamentais:

- O raio padrão de busca deve ser 3 km.
- Ao cadastrar um pet perdido, todos os usuários dentro do raio definido devem receber PUSH e notificação interna.
- A localização exata do tutor não deve ser exposta.
- O envio de notificações deve ser assíncrono via evento/fila.
- O sistema deve registrar timeline dos casos.
- O tutor pode marcar o pet como encontrado.
- Usuários podem enviar relatos de avistamento.
- Admin pode moderar casos e usuários.
- O app deve ser mobile-first, simples, rápido e emocionalmente acolhedor.
- O código deve ser limpo, modular, bem separado por domínios e com boas práticas de segurança.

Crie primeiro a estrutura do projeto, depois implemente os módulos em etapas, começando por autenticação, usuários, localização, cadastro de caso perdido, busca por raio, notificações internas e PUSH.